Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Linux ortamında C programından assembly yordamı çağırma
  - C tarafında normal bir `main` fonksiyonu yazılır ve dışarıda tanımlanacak assembly yordamı için prototip bildirilir.
  - Örnekte kullanıcıdan `n` değeri alınır, `1` değerinden `n` değerine kadar eleman içeren bir `int` dizisi oluşturulur ve bu dizi assembly tarafındaki toplama yordamına gönderilir.
  - C kodunda assembly yordamını çağırabilmek için yordamın dönüş tipi ve parametreleri doğru tanımlanmalıdır; örnek yapı `extern int topla(int *, int)` biçimindedir.
- NASM sözdizimiyle assembly dosyası yazma
  - Microsoft Assembler tarafında kullanılan `segment` yaklaşımına karşılık NASM tarafında `section` kullanılır.
  - Kod bölümü `section .text` ile tanımlanır.
  - Assembly yordamının C tarafından görülebilmesi için yordam adı `global` bildirimiyle dışa açılır.
  - C tarafında çağrılan fonksiyon adı ile assembly tarafındaki etiket adı birebir aynı olmalıdır.
- C ile assembly arasında parametre aktarımı
  - C fonksiyon çağrısında parametreler stack üzerinden aktarılır.
  - Parametreler çağrı sırasında belirli bir sırayla stack üzerine yerleşir; assembly yordamı bu değerlere `EBP` tabanlı göreli adresleme ile erişir.
  - Yordam başında `EBP` değeri korunur ve `EBP`, mevcut `ESP` değerine eşitlenerek stack frame kurulur.
  - Yordam içinde kullanılacak registerlar çağıran kodun durumunu bozmamak için `push` ile saklanır ve işlem sonunda `pop` ile geri alınır.
- Assembly yordamında dizi toplama işlemi
  - Dizinin başlangıç adresi bir registerda, eleman sayısı başka bir registerda tutulur.
  - Döngü içinde her eleman okunur, toplam değere eklenir ve dizi adresi bir sonraki `int` elemana geçecek biçimde artırılır.
  - `int` elemanlar 32 bitlik olduğu için dizi üzerinde ilerlerken adres artışı 4 bayt üzerinden düşünülür.
  - Toplam sonucunun dönüş değeri olarak kullanılabilmesi için sonuç `EAX` registerında bırakılır.
- Fonksiyon dönüş değerinin register üzerinden aktarılması
  - C tarafında bir fonksiyon `int` gibi bir değer döndürdüğünde bu değer assembly düzeyinde `EAX` registerı üzerinden döndürülür.
  - Bu nedenle assembly yordamında toplamın `EAX` üzerinde hesaplanması veya işlem sonunda `EAX` içine alınması gerekir.
  - `ret` komutu çalıştığında çağıran C kodu, dönüş değerini `EAX` üzerinden alır ve ilgili değişkene aktarır.
- NASM ve GCC ile 32 bitlik derleme ve linkleme
  - Assembly dosyası NASM ile nesne dosyasına çevrilir.
  - 32 bitlik nesne dosyası üretmek için NASM tarafında uygun format seçilir.
  - C kodu GCC ile 32 bitlik olarak derlenir; bunun için `-m32` parametresi kullanılır.
  - C nesne kodu ile assembly nesne kodu birlikte linklenerek çalıştırılabilir dosya üretilir.
  - Üretilen dosyanın çalıştırılabilir olması dosya adından değil, sistemdeki çalıştırılabi-

lirlik bilgisinden ve dosya izinlerinden anlaşılır.
- 64 bitlik registerlara geçiş hakkında temel notlar
  - 64 bitlik mimaride `EAX`, `EBX` gibi 32 bitlik register adlarının yanında `RAX`, `RBX` gibi 64 bitlik registerlar kullanılır.
  - 64 bitlik ortamda ayrıca `R8`, `R9`, `R10`, `R11`, `R12` gibi ek genel amaçlı registerlar da bulunur.
  - 64 bitlik kod yazarken yalnızca register adlarını değiştirmek yeterli olmayabilir; çağrı düzeni, parametre aktarımı ve derleme hedefi de dikkate alınmalıdır.
- Assembly kullanımının performans ve düşük seviye programlama açısından önemi
  - Assembly, yüksek performans gerektiren veya donanıma yakın çalışılması gereken durumlarda önem taşır.
  - Görüntü işleme, yapay zeka, gerçek zamanlı sistemler ve gömülü/düşük kaynaklı donanımlar gibi alanlarda belirli yordamların daha hızlı çalıştırılması gerekebilir.
  - Yüksek seviyeli dillerde yazılan programların çalıştırılmadan önce makine düzeyindeki komutlara dönüştüğü vurgulanır; bu nedenle assembly bilgisi bilgisayarın altta nasıl çalıştığını anlamak için temel bir araçtır.

Hocanın Özellikle Vurguladığı Kısımlar

- C tarafındaki `extern` bildirimi ile assembly tarafındaki `global` bildiriminin uyumu
  - C kodu dışarıda tanımlı bir yordamı çağıracağını `extern` ile bildirir.
  - Assembly kodu ise bu yordamı dışarıya görünür yapmak için `global` kullanır.
  - İki tarafta kullanılan yordam adı aynı olmazsa linker ilgili sembolü eşleştiremez.
- NASM sözdiziminde `section` kullanımının Microsoft Assembler'daki `segment` mantığına karşılık gelmesi
  - Farklı assembler araçları aynı temel kavramları farklı sözdizimleriyle ifade edebilir.
  - Bu nedenle assembly kodu yazarken kullanılan assemblerın sözdizimi bilinmelidir.
- Stack frame ve `EBP` tabanlı adresleme
  - Parametrelere güvenli ve düzenli biçimde erişmek için yordam başında stack frame kurulması gerekir.
  - `EBP` sabit referans noktası olarak kullanıldığında parametrelerin ve saklanan registerların stack üzerindeki konumu daha kolay takip edilir.
  - 32 bitlik ortamda değerlerin stack üzerindeki yerleşimi 4 baytlık aralıklarla düşünülmelidir.
- Kullanılan registerların korunması
  - Yordam içinde `ECX`, `EDI` gibi registerlar kullanılacaksa çağıran kodun beklediği değerlerin bozulmaması için bu registerlar saklanmalıdır.
  - `push` ve `pop` sırasının doğru yönetilmesi stack bütünlüğü açısından kritiktir.
- Dönüş değerinin `EAX` üzerinden verilmesi
  - `int` dönüşlü bir yordamın sonucu `EAX` registerında bulunmalıdır.
  - Toplama işlemi sonunda değerin `EAX` üzerinde kalması, C tarafındaki dönüş değerinin doğru alınmasını sağlar.
- 32 bitlik ve 64 bitlik hedeflerin karıştırılmaması
  - 32 bitlik registerlar ve 32 bitlik stack düzeniyle yazılan assembly kodu, buna uygun formatta derlenmeli ve C kodu da aynı hedef mimariye göre derlenmelidir.
  - NASM ve GCC tarafındaki hedef mimari uyumlu olmazsa register boyutları, çağrı düzeni veya linkleme aşamasında sorunlar ortaya çıkabilir.
- Assembly öğrenmenin temel amacı
  - Assembly yalnızca belirli örnekleri yazmak için değil, yüksek seviyeli programların makine düzeyinde nasıl temsil edildiğini anlamak için de önemlidir.
  - Performans kritik bölümlerde veya düşük kaynaklı donanımlarda assembly bilgisinin pratik değeri vardır.

Kısa Tekrar Notları

- C kodundan assembly yordamı çağırmak için C tarafında `extern` prototip, assembly tarafında `global` etiket gerekir.
- NASM'de kod bölümü genellikle `section .text` ile yazılır.
- C tarafındaki fonksiyon adı ile assembly tarafındaki yordam etiketi aynı olmalıdır.

- Parametreler stack üzerinden gelir; assembly yordamı bu parametrelere `EBP` tabanlı adresleme ile erişebilir.
- 32 bitlik ortamda `int` değerler ve adresler 4 baytlık aralıklarla ele alınır.
- Yordam içinde kullanılan registerlar `push` ile saklanıp `pop` ile geri yüklenmelidir.
- `int` dönüş değeri `EAX` registerı üzerinden döner.
- NASM ile assembly dosyası nesne dosyasına çevrilir; GCC ile C dosyası ve assembly nesne dosyası birlikte linklenir.
- 32 bitlik örnekte NASM formatı ve GCC `-m32` hedefi uyumlu olmalıdır.
- 64 bitlik mimaride `RAX`, `RBX` gibi registerlar ve ek `R8`-`R12` benzeri registerlar bulunur.
- Assembly, performans kritik ve donanıma yakın uygulamalarda önemli bir araçtır.

Detaylı Açıklamalar

- Bu derste Linux ortamında bir C programının NASM ile yazılmış bir assembly yordamını çağırması ele alınır. Örnek senaryoda C programı kullanıcıdan bir `n` değeri alır, bu değere kadar artan sayılardan oluşan bir `int` dizisi üretir ve diziyi toplaması için assembly yordamına gönderir. C tarafında assembly yordamı normal bir fonksiyon gibi çağrılır; ancak derleyiciye bu fonksiyonun başka bir dosyada tanımlı olduğunu bildirmek için `extern` prototipi gerekir.
- C ile assembly dosyalarının birlikte çalışabilmesi için sembol adlarının doğru eşleşmesi gerekir. C kodunda çağrılan fonksiyon `topla` ise NASM dosyasında da `topla:` etiketi bulunmalı ve bu etiket `global topla` ile dışa açılmalıdır. Bu bildirim yapılmazsa linker, C kodundaki fonksiyon çağrısını assembly dosyasındaki yordamla ilişkilendiremez.
- NASM sözdiziminde program bölümleri `section` ile tanımlanır. Microsoft Assembler tarafında kullanılan `segment` yaklaşımıyla aynı temel fikri taşır; ancak yazım biçimi farklıdır. Kodların yer aldığı bölüm `section .text` olarak belirtilir. Bu bölüm içinde yordam etiketi tanımlanır ve yordamın komutları bu etiketin altında yazılır.
- Assembly yordamı çağrıldığında parametreler stack üzerinde bulunur. Yordam başında mevcut `EBP` değeri saklanır ve `EBP`, `ESP` değerine eşitlenerek stack frame oluşturulur. Böylece parametrelere ve saklanan değerlere sabit ofsetlerle erişilebilir. 32 bitlik ortamda her adres veya `int` değer 4 bayt kabul edildiği için stack üzerindeki ilerleme 4 baytlık aralıklarla düşünülür.
- Örnekte dizi adresi ve eleman sayısı stack üzerinden alınır. Dizi adresi bir registerda, eleman sayısı ise döngü sayacı olarak kullanılabilecek bir registerda tutulur. Toplama işlemi sırasında `EAX` başlangıçta sıfırlanır, her döngü adımında dizinin o anki elemanı `EAX` üzerine eklenir ve dizi adresi bir sonraki elemana geçecek biçimde artırılır. Elemanlar `int` türünde olduğu için adres artışı 4 bayttır.
- Fonksiyon dönüş değeri konusu özellikle önemlidir. C tarafında `int` döndüren bir fonksiyonun sonucu assembly düzeyinde `EAX` registerı üzerinden iletilir. Bu nedenle assembly yordamı tamamlandığında toplam değerinin `EAX` üzerinde bulunması gerekir. `ret` komutu çağıran koda geri döndüğünde C tarafındaki değişken bu registerdan gelen değeri alır.
- Yordam içinde kullanılan registerlar çağıran kodun beklediği değerleri bozabileceği için saklanmalıdır. Ders örneğinde döngü ve dizi adresi için kullanılan registerlar yordam başında stack üzerine atılır, yordam sonunda ters sırayla geri alınır. Bu disiplin, özellikle C ve assembly kodlarının birlikte çalıştığı durumlarda programın kararlı davranması için gereklidir.
- Derleme sürecinde assembly dosyası önce NASM ile nesne dosyasına çevrilir. C dosyası GCC ile derlenirken hedefin 32 bitlik olması için `-m32` parametresi kullanılır. Sonrasında C kodu ile assemblyden üretilen nesne dosyası birlikte linklenir ve çalıştırılabilir dosya oluşturulur. Bu aşamada C kodunun hedef mimarisi ile assembly kodunun hedef formatı uyumlu olmalıdır.
- 64 bitlik sistemlerde register adları ve olanakları genişler. `EAX` gibi 32 bitlik registerların yanında `RAX` gibi 64 bitlik karşılıklar ve `R8`, `R9`, `R10` gibi ek registerlar bulunur. Ancak 64 bitlik kod yazarken yalnızca register adlarını büyütmek yeterli görülmemelidir; çağrı kuralları, parametrelerin hangi register veya stack alanları üzerinden aktarılacağı ve derleme hedefi de birlikte değerlendirilmelidir.

- Dersin genel vurgusu, assembly dilinin hem bilgisayarın alt seviyede nasıl çalıştığını anlamak hem de gerektiğinde performans kritik bölümleri optimize etmek için önemli olduğudur. Yüksek seviyeli dillerle yazılan programlar sonuçta makine komutlarına dönüştürülür. Bu dönüşümü anlamak; gömülü sistemler, gerçek zamanlı uygulamalar, yapay zeka, görüntü işleme veya düşük kaynaklı donanımlar gibi alanlarda daha bilinçli programlama yapılmasını sağlar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.